

Plano de Desenvolvimento Completo

Trabalho Pratico de Engenharia de Dados - 2026.1

CRUD Relacional, CRUD NoSQL e Integracao de Dados com Esquema Estrela

Disciplina: Engenharia de Dados

Professor: Andre Britto de Carvalho

Stack recomendada: C# + ASP.NET Core Web API + PostgreSQL + MongoDB + Apache Hop + AWS

Documento de planejamento para desenvolvimento, implementacao, testes, documentacao e apresentacao final.

1. Visao Geral do Projeto

Este plano organiza o desenvolvimento completo do Trabalho Pratico de Engenharia de Dados 2026.1. O projeto envolve tres entregas principais: uma aplicacao CRUD relacional usando PostgreSQL, uma aplicacao CRUD NoSQL usando MongoDB e uma etapa de integracao de dados com esquema estrela e pipelines de ETL no Apache Hop.

Fase	Nome	Resultado esperado
Parte 1	CRUD Relacional	Aplicacao conectada ao PostgreSQL hospedado na AWS, com CRUD para usuario, estudante, vinculo e curso.
Parte 2	CRUD NoSQL	Mapeamento das estruturas relacionais para MongoDB hospedado na AWS e implementacao do CRUD equivalente.
Parte 3	Integracao de Dados	Construcao de esquema estrela e pipelines de ETL no Apache Hop para carregar dimensoes e tabela fato.

Decisao recomendada: desenvolver o backend em C# com ASP.NET Core Web API. Essa escolha permite uma arquitetura profissional, boa demonstracao em apresentacao, integracao forte com PostgreSQL via Entity Framework Core e uso direto do MongoDB.Driver para a Parte 2.

2. Escopo e Entregaveis

2.1 Entregaveis obrigatorios

- Codigo-fonte organizado em repositorio GitHub ou ferramenta similar.
- Banco PostgreSQL hospedado na AWS contendo as tabelas do esquema relacional.
- CRUD relacional para usuario, estudante, vinculo e curso.
- Banco MongoDB hospedado na AWS contendo as estruturas mapeadas.
- CRUD NoSQL para as estruturas correspondentes.
- Discussao sobre restricoes no NoSQL: chave, integridade referencial, dominio e campos obrigatorios.
- Banco do esquema estrela em database no servidor RDS da AWS.
- Pipelines de ETL no Apache Hop para carregar dimensoes e tabela fato.
- Evidencias de execucao: requisicoes, respostas, consultas no banco, prints e logs.
- Apresentacao final demonstrando os efeitos no banco de dados.

2.2 Fora do escopo inicial

- Interface grafica completa, a menos que o grupo deseje adicionar como melhoria.
- Autenticacao complexa com JWT, permissoes e perfis avancados. Pode ser melhoria futura.
- Deploy publico de frontend. O requisito principal e demonstrar CRUD, bancos e ETL.
- Data warehouse altamente otimizado. O foco e modelagem estrela e carga correta dos dados.

3. Arquitetura Recomendada

3.1 Stack principal

Camada	Tecnologia	Justificativa
Linguagem	C#	Boa para API REST, tipagem forte, organizacao e apresentacao profissional.
Framework	ASP.NET Core Web API	Permite criar controllers, services, repositories, DTOs e validacoes.
ORM Relacional	Entity Framework Core	Facilita CRUD e mapeamento para

		PostgreSQL.
Driver PostgreSQL	Npgsql	Provider oficial/mais utilizado no ecossistema .NET para PostgreSQL.
Driver MongoDB	MongoDB.Driver	Permite CRUD direto nas colecoes MongoDB.
Banco Relacional	PostgreSQL no AWS RDS	Atende ao requisito de SGBD relacional hospedado na AWS.
Banco NoSQL	MongoDB hospedado na AWS	Atende ao requisito de banco NoSQL mapeado a partir do relacional.
ETL	Apache Hop	Ferramenta obrigatoria para pipelines de integracao da Parte 3.
Documentacao/Teste	Postman ou Insomnia	Facilita demonstrar as operacoes e gerar evidencias.

3.2 Arquitetura em camadas

- Controllers: recebem requisicoes HTTP e retornam respostas da API.
- DTOs: definem os dados de entrada e saida da API sem expor diretamente as entidades.
- Services: concentram regras de negocio, validacoes e coordenacao dos repositories.
- Repositories: realizam comunicacao com PostgreSQL ou MongoDB.
- Models/Entities: representam as entidades relacionais e documentos NoSQL.
- Data/DbContext: configuracao do Entity Framework Core para PostgreSQL.
- Configurations: conexoes, injecao de dependencias, CORS, Swagger e configuracoes gerais.

```

backend/
├── Controllers/
├── DTOs/
├── Models/
├── Services/
├── Repositories/
├── Data/
├── Mongo/
├── Configurations/
├── Program.cs
└── appsettings.json

```

4. Estrutura do Repositorio

```

engenharia-dados-crud-etl/
├── backend/
│   ├── EngenhariaDados.Api/
│   │   ├── Controllers/
│   │   ├── DTOs/
│   │   ├── Models/
│   │   ├── Services/
│   │   ├── Repositories/
│   │   ├── Data/
│   │   ├── Mongo/
│   │   ├── Program.cs
│   │   └── appsettings.example.json
│   └── README.md
├── database/
│   ├── postgresql/
│   │   ├── schema.sql
│   │   ├── seed.sql
│   │   └── star_schema.sql
│   └── mongodb/
│       └── collections.md

```

```

├── validators.js
├── etl/
│   ├── pipelines/
│   │   ├── 01_carga_departamentos.hpl
│   │   ├── 02_carga_disciplinas.hpl
│   │   ├── 03_carga_professores.hpl
│   │   ├── 04_carga_semestres.hpl
│   │   ├── 05_carga_campus.hpl
│   │   └── 06_carga_fato_turma.hpl
│   └── workflows/
│       └── workflow_carga_dw.hwf
├── data/
│   ├── unidades_academicas/
│   ├── componentes_curriculares/
│   ├── docentes/
│   └── turmas/
├── docs/
│   ├── mapeamento-nosql.md
│   ├── esquema-estrela.md
│   ├── etl-apache-hop.md
│   ├── evidencias.md
│   └── roteiro-apresentacao.md
├── README.md
└── .gitignore

```

Nunca versionar senhas reais. Usar appsettings.example.json com campos de exemplo e manter appsettings.json real fora do GitHub usando .gitignore.

5. Plano da Parte 1 - CRUD Relacional

5.1 Objetivo

Construir uma aplicacao capaz de se conectar ao PostgreSQL hospedado na AWS e executar operacoes de insercao, leitura, atualizacao e exclusao nas tabelas usuario, estudante, curso e vinculo.

5.2 Modelo relacional inicial

O modelo abaixo e uma proposta inicial. Ele deve ser ajustado conforme o esquema trabalhado em aula, caso o professor tenha fornecido nomes de colunas, chaves ou restricoes diferentes.

```

CREATE TABLE usuario (
    id_usuario SERIAL PRIMARY KEY,
    nome VARCHAR(120) NOT NULL,
    email VARCHAR(120) NOT NULL UNIQUE,
    senha VARCHAR(255) NOT NULL,
    tipo_usuario VARCHAR(50) NOT NULL
);

CREATE TABLE estudante (
    id_estudante SERIAL PRIMARY KEY,
    matricula VARCHAR(30) NOT NULL UNIQUE,
    id_usuario INTEGER NOT NULL REFERENCES usuario(id_usuario),
    data_nascimento DATE
);

CREATE TABLE curso (
    id_curso SERIAL PRIMARY KEY,
    nome VARCHAR(150) NOT NULL,
    codigo VARCHAR(30) NOT NULL UNIQUE,

```

```

    carga_horaria INTEGER NOT NULL CHECK (carga_horaria > 0)
);

CREATE TABLE vinculo (
    id_vinculo SERIAL PRIMARY KEY,
    id_estudante INTEGER NOT NULL REFERENCES estudante(id_estudante),
    id_curso INTEGER NOT NULL REFERENCES curso(id_curso),
    data_inicio DATE NOT NULL,
    status VARCHAR(30) NOT NULL,
    UNIQUE (id_estudante, id_curso)
);

```

5.3 Funcionalidades por entidade

Entidade	CRUD esperado	Validacoes principais
Usuario	Criar, listar, buscar por id, atualizar e remover usuario.	Validar nome, email unico, senha e tipo.
Estudante	Criar, listar, buscar por id, atualizar e remover estudante.	Validar matricula unica e usuario existente.
Curso	Criar, listar, buscar por id, atualizar e remover curso.	Validar codigo unico e carga horaria positiva.
Vinculo	Criar, listar, buscar por id, atualizar e remover vinculo.	Validar estudante existente, curso existente e duplicidade.

5.4 Endpoints REST recomendados

Metodo	Rota	Finalidade
POST	/api/usuarios	Cadastrar usuario
GET	/api/usuarios	Listar usuarios
GET	/api/usuarios/{id}	Buscar usuario por id
PUT	/api/usuarios/{id}	Atualizar usuario
DELETE	/api/usuarios/{id}	Remover usuario
POST	/api/estudantes	Cadastrar estudante
GET	/api/estudantes	Listar estudantes
POST	/api/cursos	Cadastrar curso
GET	/api/cursos	Listar cursos
POST	/api/vinculos	Cadastrar vinculo
GET	/api/vinculos	Listar vinculos

5.5 Etapas de desenvolvimento da Parte 1

1. Criar instancia PostgreSQL no AWS RDS.
2. Liberar grupo de seguranca para acesso controlado ao banco.
3. Criar database da aplicacao.
4. Executar schema.sql e seed.sql.
5. Criar projeto ASP.NET Core Web API.
6. Configurar Entity Framework Core e Npgsql.
7. Criar entidades, DbContext e migrations, se necessario.
8. Criar repositories e services.
9. Criar controllers REST.
10. Testar cada endpoint no Postman/Insomnia.
11. Registrar evidencias das alteracoes no PostgreSQL.

5.6 Critérios de aceite da Parte 1

- A aplicação conecta corretamente ao PostgreSQL da AWS.
- Todas as tabelas obrigatórias existem no banco.
- Cada entidade possui create, read, update e delete funcionais.
- As restrições do banco impedem dados inválidos.
- As evidências mostram o efeito de cada método no banco.

6. Plano da Parte 2 - CRUD NoSQL

6.1 Objetivo

Mapear as estruturas do modelo relacional para o MongoDB e implementar o CRUD das coleções correspondentes a usuário, estudante, curso e vínculo.

6.2 Estratégia de mapeamento

Como o trabalho exige representar todas as tabelas no NoSQL, a abordagem mais segura para avaliação e criar coleções equivalentes às tabelas relacionais, usando referências por ObjectId. Essa estratégia facilita demonstrar a correspondência entre os dois modelos e discutir integridade referencial na camada da aplicação.

Tabela relacional	Coleção MongoDB	Campos principais
usuario	usuarios	_id, nome, email, senha, tipo_usuario
estudante	estudantes	_id, matricula, usuario_id, data_nascimento
curso	cursos	_id, nome, codigo, carga_horaria
vinculo	vinculos	_id, estudante_id, curso_id, data_inicio, status

6.3 Documentos sugeridos

```
// usuarios
{
  "_id": ObjectId("..."),
  "nome": "Maria Silva",
  "email": "maria@email.com",
  "senha": "hash_da_senha",
  "tipo_usuario": "estudante"
}

// estudantes
{
  "_id": ObjectId("..."),
  "matricula": "20260001",
  "usuario_id": ObjectId("..."),
  "data_nascimento": ISODate("2000-01-01")
}

// cursos
{
  "_id": ObjectId("..."),
  "nome": "Engenharia de Computacao",
  "codigo": "ECOMP",
  "carga_horaria": 3600
}

// vinculos
{
  "_id": ObjectId("..."),
```

```

"estudante_id": ObjectId("..."),
"curso_id": ObjectId("..."),
"data_inicio": ISODate("2026-01-01"),
"status": "ativo"
}

```

6.4 Como tratar restricoes no MongoDB

Restricao	Tratamento proposto
Chave	Uso do campo _id e indices unique para campos como email, matricula e codigo.
Integridade referencial	Validar na aplicacao se usuario_id, estudante_id e curso_id existem antes de inserir ou atualizar.
Dominio	Usar validacoes no service e, quando possivel, JSON Schema Validator no MongoDB.
NOT NULL	Usar campos required no validator e validacoes de DTO na API.

6.5 Etapas de desenvolvimento da Parte 2

12. Criar ambiente MongoDB hospedado na AWS.
13. Criar database NoSQL do projeto.
14. Definir colecoes e validators.
15. Criar indices unicos para email, matricula e codigo de curso.
16. Configurar MongoDB.Driver no projeto C#.
17. Criar models/documentos MongoDB.
18. Criar repositories MongoDB.
19. Criar services com validacoes de integridade.
20. Criar controllers ou rotas separadas para a Parte 2.
21. Testar CRUD no Postman/Insomnia.
22. Registrar evidencias no MongoDB Compass ou shell.

6.6 Criterios de aceite da Parte 2

- Todas as tabelas relacionais possuem representacao no MongoDB.
- O CRUD NoSQL funciona para usuario, estudante, curso e vinculo.
- As restricoes exigidas foram discutidas e implementadas na aplicacao ou no banco.
- As evidencias mostram o efeito de cada metodo no MongoDB.

7. Plano da Parte 3 - Integracao de Dados

7.1 Objetivo

Construir um banco em esquema estrela no PostgreSQL/RDS e desenvolver pipelines no Apache Hop para carregar dimensoes e tabela fato a partir do banco relacional da Parte 1 e dos arquivos CSV do grupo Ensino do dados.ufs.br.

7.2 Fontes de dados

Fonte	Tipo	Uso
Banco relacional da Parte 1	PostgreSQL AWS RDS	Linhas originais do script e linhas inseridas pelo CRUD.
Unidades Academicas	CSV dados.ufs.br	Base para departamentos/unidades.
Componentes Curriculares	CSV dados.ufs.br	Base para disciplinas.
Docentes	CSV dados.ufs.br	Base para professores.
Turmas	CSV dados.ufs.br	Base para semestre, campus e fato

		turma. Considerar 2019 a 2025.
--	--	--------------------------------

7.3 Dimensoes e tabela fato

Tabela	Representa	Campos/metricas principais
dim_professor	Professores	nome, tipo_jornada, formacao, departamento_lotacao
dim_disciplina	Componentes curriculares	codigo, nome, departamento_responsavel, cr_total
dim_departamento	Departamentos/unidades	codigo/sigla, nome
dim_semestre	Semestres	ano, periodo
dim_campus	Campus	nome do campus
fato_turma	Turmas de graduacao	matriculados, media_notas, aprovados, reprovados

7.4 Schema estrela sugerido

```

CREATE TABLE dim_professor (
  id_professor SERIAL PRIMARY KEY,
  nome VARCHAR(255) NOT NULL,
  tipo_jornada VARCHAR(100),
  formacao VARCHAR(100),
  departamento_lotacao VARCHAR(100)
);

CREATE TABLE dim_disciplina (
  id_disciplina SERIAL PRIMARY KEY,
  codigo VARCHAR(50) NOT NULL,
  nome VARCHAR(255) NOT NULL,
  departamento_responsavel VARCHAR(100),
  cr_total INTEGER
);

CREATE TABLE dim_departamento (
  id_departamento SERIAL PRIMARY KEY,
  codigo VARCHAR(50) NOT NULL,
  nome VARCHAR(255) NOT NULL
);

CREATE TABLE dim_semestre (
  id_semestre SERIAL PRIMARY KEY,
  ano INTEGER NOT NULL,
  periodo INTEGER NOT NULL,
  UNIQUE (ano, periodo)
);

CREATE TABLE dim_campus (
  id_campus SERIAL PRIMARY KEY,
  nome VARCHAR(255) NOT NULL UNIQUE
);

CREATE TABLE fato_turma (
  id_fato_turma SERIAL PRIMARY KEY,
  id_professor INTEGER REFERENCES dim_professor(id_professor),
  id_disciplina INTEGER REFERENCES dim_disciplina(id_disciplina),
  id_departamento INTEGER REFERENCES dim_departamento(id_departamento),
  id_semestre INTEGER REFERENCES dim_semestre(id_semestre),
  id_campus INTEGER REFERENCES dim_campus(id_campus),

```

```

numero_discentes_matriculados INTEGER,
media_notas NUMERIC(5,2),
numero_aprovados INTEGER,
numero_reprovados INTEGER
);

```

7.5 Pipelines no Apache Hop

Pipeline	Fonte	Destino	Transformacoes
01_carga_departamentos	CSV Unidades Academicas	dim_departamento	Selecionar codigo/sigla e nome, remover duplicidades.
02_carga_disciplinas	CSV Componentes Curriculares	dim_disciplina	Selecionar codigo, nome, departamento e creditos.
03_carga_professores	CSV Docentes	dim_professor	Selecionar nome, jornada, formacao e departamento.
04_carga_semestres	CSV Turmas	dim_semestre	Extrair ano e periodo, filtrar 2019 a 2025.
05_carga_campus	CSV Turmas	dim_campus	Extrair e padronizar campus.
06_carga_fato_turma	CSV Turmas + dimensoes + banco Parte 1	fato_turma	Fazer lookup das dimensoes e inserir metricas.

7.6 Ordem de execucao

23. Limpar ou garantir que as tabelas do esquema estrela estejam vazias antes da avaliacao.
24. Executar cargas das dimensoes: departamentos, disciplinas, professores, semestres e campus.
25. Validar contagem de registros de cada dimensao.
26. Executar carga da tabela fato turma.
27. Validar integridade dos relacionamentos por meio de joins.
28. Executar consultas analiticas para demonstracao final.

7.7 Consultas de validacao

```

SELECT COUNT(*) FROM dim_professor;
SELECT COUNT(*) FROM dim_disciplina;
SELECT COUNT(*) FROM dim_departamento;
SELECT COUNT(*) FROM dim_semestre;
SELECT COUNT(*) FROM dim_campus;
SELECT COUNT(*) FROM fato_turma;

SELECT
  s.ano,
  s.periodo,
  d.nome AS disciplina,
  p.nome AS professor,
  f.numero_discentes_matriculados,
  f.media_notas,
  f.numero_aprovados,
  f.numero_reprovados
FROM fato_turma f
JOIN dim_semestre s ON s.id_semestre = f.id_semestre
JOIN dim_disciplina d ON d.id_disciplina = f.id_disciplina
JOIN dim_professor p ON p.id_professor = f.id_professor
ORDER BY s.ano, s.periodo, d.nome;

```

8. Plano de Ambiente e AWS

8.1 Recursos necessarios

Recurso	Uso
AWS RDS PostgreSQL	Banco relacional da Parte 1 e banco do esquema estrela da Parte 3.
MongoDB na AWS	Banco NoSQL da Parte 2. Pode ser instalado em EC2 ou usar servico compativel.
Security Groups	Liberar acesso apenas para IPs necessarios durante desenvolvimento e apresentacao.
Usuarios de banco	Criar usuario da aplicacao e usuario especifico das rotinas de ETL.
Backups/snapshots	Evitar perda de dados antes da apresentacao.

8.2 Boas praticas de configuracao

- Nunca publicar senha no GitHub.
- Usar arquivo appsettings.example.json para documentar variaveis de conexao.
- Criar usuario especifico para ETL com permissoes de INSERT, SELECT e, se necessario, DELETE/TRUNCATE nas tabelas do esquema estrela.
- Registrar os comandos SQL usados para criar usuarios, schemas e permissoes.
- Testar conexao do Apache Hop com antecedencia, no mesmo padrao que sera usado na apresentacao.

9. Cronograma de Desenvolvimento

Periodo	Foco	Entregas
Semana 1	Planejamento e ambiente	Repositorio, README, AWS RDS, MongoDB, estrutura do projeto, schema inicial.
Semana 2	CRUD Relacional 1	Entidades Usuario e Curso, conexao PostgreSQL, endpoints e testes.
Semana 3	CRUD Relacional 2	Entidades Estudante e Vinculo, validacoes, evidencias e ajustes.
Semana 4	Mapeamento NoSQL	Pesquisa, definicao das colecoes, validators e indices.
Semana 5	CRUD NoSQL	Repositories MongoDB, services, controllers, testes e evidencias.
Semana 6	Esquema estrela	Criar schema dimensional no RDS, tabelas de dimensao e fato.
Semana 7	Pipelines ETL	Construir pipelines no Apache Hop e testar cargas com CSVs.
Semana 8	Revisao final	Executar cenario completo, preparar relatorio, prints e roteiro de apresentacao.

10. Kanban e Issues no GitHub

O desenvolvimento deve ser acompanhado por tarefas pequenas e verificaveis. Abaixo esta uma sugestao de issues para organizar o projeto.

Issue	Tarefa	Modulo	Prioridade
#1	Criar estrutura inicial do	Planejamento	Alta

	repositorio		
#2	Criar schema relacional PostgreSQL	Parte 1	Alta
#3	Configurar ASP.NET Core com PostgreSQL	Parte 1	Alta
#4	Implementar CRUD de usuarios	Parte 1	Alta
#5	Implementar CRUD de estudantes	Parte 1	Alta
#6	Implementar CRUD de cursos	Parte 1	Alta
#7	Implementar CRUD de vinculos	Parte 1	Alta
#8	Documentar evidencias PostgreSQL	Parte 1	Media
#9	Definir mapeamento NoSQL	Parte 2	Alta
#10	Criar colecoes e indices MongoDB	Parte 2	Alta
#11	Implementar CRUD NoSQL	Parte 2	Alta
#12	Documentar restricoes no MongoDB	Parte 2	Media
#13	Criar schema estrela	Parte 3	Alta
#14	Construir pipelines de dimensoes	Parte 3	Alta
#15	Construir pipeline da fato_turma	Parte 3	Alta
#16	Preparar roteiro de apresentacao	Final	Alta

11. Plano de Testes e Evidencias

11.1 Testes de CRUD

- Para cada POST, demonstrar o registro criado no banco.
- Para cada GET, demonstrar retorno de lista e busca por id.
- Para cada PUT, demonstrar dados antes e depois da atualizacao.
- Para cada DELETE, demonstrar que o registro foi removido ou marcado como removido.
- Testar cenarios invalidos: email duplicado, matricula duplicada, curso inexistente, estudante inexistente e campos obrigatorios ausentes.

11.2 Evidencias por pasta

```
docs/evidencias/
├── parte1-postgresql/
│   ├── usuarios.md
│   ├── estudantes.md
│   ├── cursos.md
│   └── vinculos.md
├── parte2-mongodb/
│   ├── usuarios.md
│   ├── estudantes.md
│   ├── cursos.md
│   └── vinculos.md
└── parte3-etl/
```

11.3 Evidencias da Parte 3

- Print das tabelas do esquema estrela vazias antes da execucao.
- Print ou log da execucao dos pipelines no Apache Hop.
- Consultas COUNT depois da carga.
- Consulta com joins entre fato e dimensoes.
- Explicacao de transformacoes relevantes: limpeza, deduplicacao, filtro de ano e lookup de dimensoes.

12. Roteiro da Apresentacao Final

Ordem	Momento	O que demonstrar
1	Contexto do projeto	Explicar as tres partes do trabalho e a stack utilizada.
2	Parte 1	Mostrar PostgreSQL na AWS, tabelas e CRUD relacional funcionando.
3	Parte 2	Mostrar mapeamento para MongoDB, colecoes, restricoes e CRUD NoSQL.
4	Parte 3	Mostrar esquema estrela, pipelines Apache Hop e carga das tabelas vazias.
5	Evidencias	Executar consultas no banco e mostrar os efeitos das operacoes.
6	Conclusao	Apresentar dificuldades, decisoes e melhorias futuras.

12.1 Demonstracao minima obrigatoria

- Inserir um usuario, estudante, curso e vinculo no PostgreSQL.
- Consultar esses registros diretamente no banco.
- Inserir documentos equivalentes no MongoDB.
- Mostrar validacao de alguma restricao no MongoDB ou na aplicacao.
- Executar pelo menos um workflow ou os pipelines no Apache Hop.
- Mostrar as tabelas do esquema estrela preenchidas apos a execucao.

13. Riscos e Mitigacoes

Risco	Impacto	Mitigacao
Conexao AWS bloqueada	Alta	Testar security group com antecedencia e ter IP atualizado.
Senha exposta no GitHub	Alta	Usar .gitignore e appsettings.example.json sem credenciais reais.
MongoDB sem integridade referencial	Media	Implementar validacoes nos services e documentar a estrategia.
CSVs com colunas inesperadas	Alta	Inspecionar arquivos antes e padronizar nomes no Apache Hop.
Pipelines falhando na apresentacao	Alta	Testar com tabelas vazias antes e salvar logs de execucao.
Falta de evidencias	Media	Criar docs/evidencias desde o inicio.

14. Melhorias Futuras

- Adicionar autenticação com JWT e controle de perfis.
- Criar frontend simples para demonstrar os CRUDs sem depender apenas do Postman.
- Adicionar testes automatizados unitários e de integração.
- Criar dashboard analítico conectado ao esquema estrela.
- Automatizar a execução dos pipelines em workflow único no Apache Hop.
- Adicionar Docker Compose para ambiente local de desenvolvimento.

15. Checklist Final

Modulo	Item	Status
Parte 1	PostgreSQL AWS configurado	Pendente
Parte 1	CRUD relacional completo	Pendente
Parte 1	Evidências PostgreSQL registradas	Pendente
Parte 2	MongoDB AWS configurado	Pendente
Parte 2	Mapeamento NoSQL documentado	Pendente
Parte 2	CRUD NoSQL completo	Pendente
Parte 3	Schema estrela criado no RDS	Pendente
Parte 3	Pipelines Apache Hop criados	Pendente
Parte 3	Carga testada com tabelas vazias	Pendente
Final	Relatório e README atualizados	Pendente
Final	Roteiro de apresentação pronto	Pendente

16. Proximos Passos Recomendados

29. Confirmar com o professor ou material da aula o schema relacional exato das tabelas usuário, estudante, vínculo e curso.
30. Definir oficialmente a stack do grupo. Recomendação: C# + ASP.NET Core Web API.
31. Criar o repositório GitHub e subir README, .gitignore e estrutura inicial.
32. Criar banco PostgreSQL no RDS e testar conexão local.
33. Implementar primeiro o CRUD relacional, pois ele também será fonte para a Parte 3.
34. Em paralelo, baixar e analisar os CSVs do dados.ufs.br para antecipar problemas da integração.